

Writing Simple Programs (Part 1)

CMSC 104 Section 2
31 August 2026

Administrative Notes

Classwork 1 is due this Wednesday, 2 September before midnight

Classwork 2 will be released this Wednesday

There is no class next Monday, 7 September - it's Labor Day; enjoy your holiday

So what did we cover last week?

A key point: in this class, we're writing application software

Cool, but what's "application software" and where does it fit in the "computer" ecology?

Hardware, software, system software, application software

Generally, we consider a computer system to include:

- A Central Processing Unit (CPU) with control, arithmetic&logic, and memory
- Input devices such as keyboards, mice,...
- Output devices such as monitors, printers,
- And maybe other stuff: additional memory/storage (an SSD),...

If you can physically touch it, it's hardware

If it's the logic that makes things happen, it's software

If it controls the operations of the computer, it's system software

If it does useful or entertaining stuff, it's application software

One more topic I skipped last week: high-level languages

Remember the bit: computers only understand 1's and 0's.

When computers were first built (1940s/1950s) you could only write software in 1's and 0's - you had to translate that yourself

- The dirty secret: that's what computers still execute today. The most powerful LLM is nothing but a bunch of 1s and 0s.

To make it easier on the human, later computers allowed assembly language programming (see https://en.wikipedia.org/wiki/Assembly_language)

“Hello, world” in Assembly Language

```
hello:
    mov     esi,msg      ; address of string into ESI
    cld                ; Set direction to increment ESI
    lodsb             ; Load first char in AL, inc ESI
chr1p:
    call outchr         ; Print character in AL
    lodsb             ; Load next character in AL, inc ESI
    or al, al          ; Is it a zero terminator?
    bne chr1p          ; If not, continue
    ret                ; Return to caller

msg: db 'Hello, world!', 0xa, 0x0 ; string to be printed
```

The program
that turns this
into machine
code is called
an
“assembler”

That's gross - hard to learn; easy to make mistakes

So the solution is “high level languages”

- High level languages were written to be “user-friendly”
- Easily understood by a competent human
- Standard across different computers and computer types

Examples:

- FORTRAN
- COBOL
- C
- C++
- Java
- Python
- Javascript
- GO
- PERL

All languages are essentially equal

Alan Turing's PhD dissertation essentially proved that all high-level computer languages are equal

- If you can solve a problem in one language, you can solve it in another, too
- It just might be easier or harder in specific languages

So why do we teach this class in Python?

- When I started teaching it we taught it in C
- Python is more general, and “easier” for non-programmer humans to read, understand and use
- It's widely used; there's a ton of free Python software out there
- Its use is growing with big data and AI

Okay, let's get to “how Python works on computers”

A bit about how computers work internally

You can think of the computer's memory as a long set of places to store bits

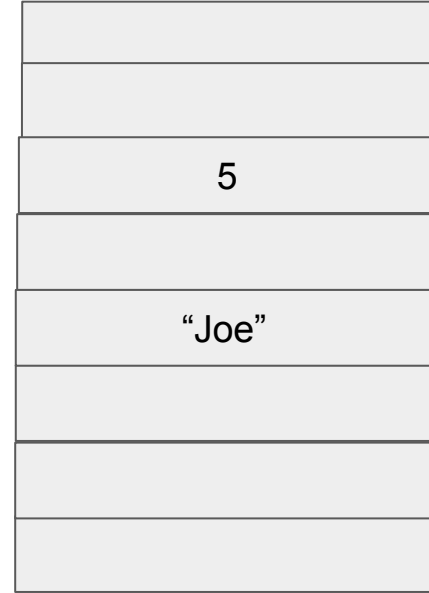
- Each place has an internal address (that we don't care about - it depends on what else is running on your computer at the time)
- How many bits can be stored depends on your computer; 32 and 64 are typical
- What those bits mean depend on what type of data we're talking about

140733399356328

5

140733399356344

"Joe"



Python helps us manage this

Python creates a “symbol table” with every identifier (variable name) you use in your program

- It allocates a location in memory
- Identifies what type it's supposed to hold (integer, real number, string,...)
- Maps your name to the location's address

When you use that variable in your program, Python goes to that address in memory and gets the value



A simplified illustration

Symbol`	Type	Address
index	integer	140733399356328
name	string	140733399356344

The fact that Python manages this
means we don't have to!!!

What's a “symbol?”

Okay, we have a “symbol table”. What's a “symbol?”

- It's any name you use in your program
- Right now, we've only learned about “variables”
 - There will be more later
- Why is it a “variable?”
 - Because its value can change in the program
- What are the rules on variable names, again?
 - Start with an underscore (`_`) or letter - upper or lower case are both fine
 - Contains underscores, upper & lower case letters, and digits
 - Can be as long as you want

So, let's look at some examples and see how it works

```
for i in range(1, 11, 1):  
    print("Hello, world")
```

```
for i in range(1, 11, 1):  
    print("the number is", i)
```